

# CSE 220: Signals and Systems Sessional

## Practice Mock Quiz Set — Fourier Series & Continuous Fourier Transform

---

This set contains four practice quizzes, written in the same style, length, and difficulty as the two online quizzes you have already completed (energy-based harmonic pruning on `fs_redrawer.py`, and band-pass/band-stop/DC filtering on `cft_edge_detector.py`). Each quiz builds directly on top of your existing, working Task 1 / Task 2 implementations — assume those methods (`calculate_cn`, `approximate`, `compute_cft`, `reconstruct`, `high_pass`, etc.) are already pasted in and working.

Attempt each quiz cold, under its stated time limit, with your Task 1/Task 2 files open but without looking at the answer key. The full answer key with explanations and code is at the end of this document.

| Quiz                                                                  | Marks |
|-----------------------------------------------------------------------|-------|
| Quiz 1 &mdash; Fourier Series: Harmonic Truncation vs. Energy Pruning | 10    |
| Quiz 2 &mdash; Fourier Series: Shape Transformation via Coefficients  | 10    |
| Quiz 3 &mdash; CFT: Quadrant Masking & Conjugate Symmetry             | 10    |
| Quiz 4 &mdash; CFT: Notch Filter & Parseval Energy Check              | 10    |

## Online — Fourier Series (Practice)

Total Marks: 10 | Time: 25 Minutes

### Context & Objective

In Task 1 of the offline assignment, a 2D drawing is reconstructed using all  $2N+1$  Fourier harmonics. A simpler (and cheaper) compression strategy than energy-based pruning is **hard truncation**: keep only the low-order harmonics  $|n| \leq k$  and discard everything above that order, regardless of how much energy each individual harmonic carries. Your task is to implement this truncation scheme in `fs_redrawer.py` and compare it against the energy-based pruning you already built.

### Task 1: Order-Based Harmonic Truncation

In the `FourierEpicycles` class, implement the method `truncate_harmonics(self, k)`:

- Given a non-negative integer order  $k \leq N$ , retain only the coefficients  $c_n$  for which  $|n| \leq k$ . Every coefficient with  $|n| > k$  must be set to zero (do not delete dictionary keys — the coefficient count must stay  $2N+1$ ).
- The method must not permanently destroy the original coefficients: repeated calls with different values of  $k$ , on the same object, must each truncate from the full, original  $2N+1$ -harmonic set — not from a previously truncated one.
- The method should return a tuple `(retained_count, energy_ratio)`, where `retained_count` is the number of non-zero coefficients kept ( $2k+1$ ), and `energy_ratio` is the fraction of total signal energy  $E_{\text{total}} = \sum |c_n|^2$  that those retained harmonics account for.

### Task 2: Compression Efficiency Comparison

In the `FourierEpicycles` class, implement the method `compare_truncation_vs_pruning(self, k)`, which compares order-based truncation against energy-based pruning (`prune_harmonics_by_energy` from the previous quiz) **at matched harmonic count**:

- Call `truncate_harmonics(k)` and record its `energy_ratio` and MSE (via your existing `evaluate_reconstruction_error`).
- Then call `prune_harmonics_by_energy(r)` using the exact energy ratio  $r$  achieved by the truncation above, and record how many harmonics energy-based pruning needed to reach that *same* ratio, plus its MSE.
- Return a dictionary with keys `trunc_count`, `trunc_mse`, `prune_count`, `prune_mse`.

### Execution & Deliverables

In the main execution block, load `svgs/heart.svg` with  $N = 150$ . For  $k \in \{5, 20, 50, 100, 150\}$ , call `compare_truncation_vs_pruning(k)` and print:

| k   | Trunc. Count | Trunc. MSE | Prune Count (same ratio) | Prune MSE |
|-----|--------------|------------|--------------------------|-----------|
| 5   | ...          | ...        | ...                      | ...       |
| 20  | ...          | ...        | ...                      | ...       |
| ... |              |            |                          |           |
| 150 | 301          | ...        | 301                      | ...       |

Save the comparison plot for  $k = 20$  as `heart_truncation_k20.png`.

## Online — Fourier Series (Practice)

Total Marks: 10 | Time: 25 Minutes

### Context & Objective

A useful property of the complex Fourier series is that simple geometric transformations of the traced shape — rotation and uniform scaling — correspond to simple algebraic operations on the coefficients  $c_n$ , with no need to re-integrate against the original signal. Your task is to implement these coefficient-domain transformations in `fs_redrawer.py` and verify them against a directly-transformed reference signal.

### Task 1: Coefficient-Domain Rotation and Scaling

In the `FourierEpicycles` class, implement two methods:

- `get_rotated_coeffs(self, theta)` — returns a *new* dictionary of coefficients corresponding to the original shape rotated counter-clockwise by angle  $\theta$  (radians) about the origin. Must not modify `self.coeffs`.
- `get_scaled_coeffs(self, factor)` — returns a *new* dictionary of coefficients corresponding to the original shape scaled uniformly about the origin by the given real factor. Must not modify `self.coeffs`.

Recall  $f(t) = \sum c_n e^{jn\omega t}$ , and that a rotation by  $\theta$  maps every point  $z$  of the drawing to  $z \cdot e^{j\theta}$ , while a scale by  $s$  maps every point  $z$  to  $s \cdot z$ .

### Task 2: Transformation Verification

In the `FourierEpicycles` class, implement `verify_transformation(self, transformed_coeffs, z_reference, t)`:

- Reconstruct  $f(t)$  using `transformed_coeffs` instead of `self.coeffs` (temporarily swap them in, then restore `self.coeffs` afterward, since later parts of the script still need the original untransformed object).
- Compute and return the MSE between this reconstruction and `z_reference`, the independently-computed ground truth (e.g. `z * np.exp(1j*theta)` for rotation, or `factor * z` for scaling).

### Execution & Deliverables

In the main execution block, load `svgs/star.svg` with  $N = 150$ . For  $\theta \in \{0, \pi/4, \pi/2, \pi\}$  verify rotation, and for `factor`  $\in \{0.5, 1.5, 2.0\}$  verify scaling. Print:

| Transform | Parameter | Verification MSE |
|-----------|-----------|------------------|
| Rotation  | 0.0000    | ...              |
| Rotation  | 0.7854    | ...              |
| Rotation  | 1.5708    | ...              |
| Rotation  | 3.1416    | ...              |
| Scaling   | 0.5       | ...              |
| Scaling   | 1.5       | ...              |
| Scaling   | 2.0       | ...              |

All verification MSE values are expected to be at the level of numerical/integration noise (very small), since rotation/scaling coefficients are algebraically exact. Save the  $\theta = \pi/2$  rotated overlay plot as `star_rotated_90.png`.

## Online — Continuous Fourier Transform (Practice)

Total Marks: 10 | Time: 30 Minutes

### Definitions

For a CFT spectrum of size  $H \times W$ , the center is  $(c_i, c_j) = (H/2, W/2)$ . For entry  $(i, j)$ , define its signed offsets  $\Delta i = i - c_i$  and  $\Delta j = j - c_j$ . The four quadrants of the spectrum (Cartesian convention,  $i$  increasing downward,  $j$  increasing rightward) are:

- Quadrant 1:  $\Delta i \leq 0, \Delta j > 0$
- Quadrant 2:  $\Delta i \leq 0, \Delta j \leq 0$
- Quadrant 3:  $\Delta i > 0, \Delta j \leq 0$
- Quadrant 4:  $\Delta i > 0, \Delta j > 0$

### Task 1: Quadrant Masking

In the `FrequencyFilter` class, implement `quadrant_mask(self, real, imag, keep_quadrants)`, where `keep_quadrants` is a set/list of integers from  $\{1, 2, 3, 4\}$ :

- Zero out every entry whose quadrant (per the Definitions above) is *not* in `keep_quadrants`; leave entries in the kept quadrants unchanged.
- Must not use Python-level nested loops over pixels — build the quadrant membership using vectorized NumPy operations on index grids.

### Task 2: Conjugate Symmetry Verification

Since the input image  $I(x, y)$  is strictly real-valued, its 2D-CFT must satisfy conjugate (Hermitian) symmetry:  $F(-u, -v) = F(u, v)^*$ , i.e.  $\Re\{F(-u, -v)\} = \Re\{F(u, v)\}$  and  $\Im\{F(-u, -v)\} = -\Im\{F(u, v)\}$ .

In the `ReconstructionValidator` class, implement `verify_conjugate_symmetry(self, real, imag)`:

- Compute the point-reflected arrays (real and imag flipped along both axes) and compare against the symmetry condition above.
- Return `(is_symmetric, max_deviation)`, where `is_symmetric` is True if the maximum absolute deviation from the symmetry condition, over both the real and imaginary parts, is below  $10^{-6}$ .
- Note: with an even-length axis, index-reversal via `np.flip` pairs each bin with the bin one index short of its true mirror partner (since there is no exact zero-frequency-centered bin). A small residual deviation is therefore expected even for a perfectly correct spectrum — this is why the tolerance is  $10^{-6}$ , not exact equality.

### Execution & Deliverables

Load `pikachu.png`. Compute the full spectrum with `compute_cft()` and verify its conjugate symmetry.

Then produce two quadrant-masked reconstructions: `keep_quadrants = {1, 3}` (diagonal pair) and `keep_quadrants = {1, 2}` (top half). For each, reconstruct with `InverseCFT2D` and save as an edge map (`abs`, `normalize`, `invert` — same convention as the provided `high_pass` pipeline). Print:

Conjugate symmetry check: True | max deviation: 3.14e-11

Quadrants Kept | Output File

```
-----+-----
{1, 3}   | pikachu_quad13.png
{1, 2}   | pikachu_quad12.png
```

## Online — Continuous Fourier Transform (Practice)

Total Marks: 10 | Time: 30 Minutes

### Definitions

Using the same center  $(c_i, c_j)$  and distance  $d(i, j)$  as in the band-pass/band-stop quiz. Parseval's theorem for the 2D CFT states that spatial-domain energy equals frequency-domain energy (up to the fixed sampling grid used here):

$$E_{\text{spatial}} = \sum_{x,y} I(x,y)^2 \approx E_{\text{freq}} = \sum_{u,v} (\Re\{F(u,v)\}^2 + \Im\{F(u,v)\}^2) \cdot k$$

where  $k$  is a fixed normalization constant determined by the sampling grid (you do not need to derive  $k$  analytically — estimate it empirically, see Task 2).

### Task 1: Notch Filter

A notch filter removes a single narrow annular ring (rather than a disk from the origin, or a broad band). In the `FrequencyFilter` class, implement `notch_filter(self, real, imag, center_radius, width)`:

- Zero out every entry  $(i, j)$  with  $\text{center\_radius} - \text{width}/2 \leq d(i,j) \leq \text{center\_radius} + \text{width}/2$ . Leave all other entries (both inside the inner radius and outside the outer radius) unchanged.
- This is functionally a band-stop filter with  $r_{\text{low}} = \text{center\_radius} - \text{width}/2$  and  $r_{\text{high}} = \text{center\_radius} + \text{width}/2$  — but implement it independently, using the  $(\text{center\_radius}, \text{width})$  parametrization directly, without calling `band_stop`.

### Task 2: Empirical Parseval Constant & Energy Conservation Check

In the `ReconstructionValidator` class, implement `check_energy_conservation(self, image, real, imag)`:

- Compute  $E_{\text{spatial}} = \sum \text{image}^2$  from the original (unfiltered) image array, and  $E_{\text{freq\_raw}} = \sum (\text{real}^2 + \text{imag}^2)$  from the *unfiltered* full spectrum.
- Compute the empirical constant  $k = E_{\text{spatial}} / E_{\text{freq\_raw}}$ .
- Given a second, filtered spectrum  $(\text{real\_f}, \text{imag\_f})$ , compute  $E_{\text{freq\_filtered}} = \sum (\text{real\_f}^2 + \text{imag\_f}^2)$ , and the fraction of original spatial energy retained:  $\text{retained\_fraction} = (E_{\text{freq\_filtered}} \cdot k) / E_{\text{spatial}}$ .
- Return  $(k, \text{retained\_fraction})$ .

This constant  $k$  should come out very close to  $(\Delta x \cdot \Delta y)^2$ , i.e. the square of the spatial-grid quadrature weight — you do not need to prove this on the quiz, just report the empirical value.

### Execution & Deliverables

Load `pikachu.png`, compute its full spectrum, and report  $k$ . Then apply `notch_filter` with `center_radius = 30`, `width = 10`, reconstruct the filtered spectrum, and report the retained energy fraction. Save the filtered reconstruction as an edge map `pikachu_notched.png`. Print:

Empirical Parseval constant  $k$ : 6.1035e-05

Notch filter (center=30, width=10) retained energy fraction: 0.9823

Saved `pikachu_notched.png`

# Answer Key & Full Explanations

Complete solutions, code, and reasoning for all four practice quizzes

---

## Quiz 1 — Harmonic Truncation vs. Energy Pruning

**Reasoning.** Truncation by order and pruning by energy are two different selection rules operating on the same set of  $2N+1$  coefficients. Truncation is *positional* (keep low  $|n|$ , regardless of size); pruning is *value-based* (keep the largest  $|c_n|^2$ , regardless of position). For a smooth, low-frequency-dominated shape the two will pick nearly the same harmonics and perform similarly. For a shape with sharp corners (like the heart's cusp), a lot of energy sits in higher-order harmonics that a fixed-order cutoff would exclude but energy-based pruning would happily keep — so at a matched harmonic count, **energy pruning's MSE should be  $\leq$  truncation's MSE**, and to hit the same energy ratio, pruning should need **fewer or equal** harmonics than truncation. If your printed table shows truncation consistently winning, double-check your energy-ratio computation.

```
def truncate_harmonics(self, k):
    # Snapshot originals once, exactly like the energy-pruning method,
    # so repeated calls always truncate from the full 2N+1 set.
    if not hasattr(self, '_original_coeffs'):
        self._original_coeffs = dict(self.coeffs)

    total_energy = sum(abs(c)**2 for c in self._original_coeffs.values())

    new_coeffs = {}
    kept_energy = 0.0
    for n, c in self._original_coeffs.items():
        if abs(n) <= k:
            new_coeffs[n] = c
            kept_energy += abs(c)**2
        else:
            new_coeffs[n] = 0j
    self.coeffs = new_coeffs

    retained_count = 2 * min(k, self.N) + 1
    energy_ratio = kept_energy / total_energy
    return retained_count, energy_ratio

def compare_truncation_vs_pruning(self, k):
    trunc_count, trunc_ratio = self.truncate_harmonics(k)
    trunc_mse = self.evaluate_reconstruction_error()

    prune_count, prune_ratio = self.prune_harmonics_by_energy(trunc_ratio)
    prune_mse = self.evaluate_reconstruction_error()

    return {
        "trunc_count": trunc_count,
        "trunc_mse": trunc_mse,
        "prune_count": prune_count,
        "prune_mse": prune_mse,
    }
```

**Key trap.** `evaluate_reconstruction_error` reads whatever is currently in `self.coeffs` — you must call it *immediately* after each pruning/truncation call, before the next one overwrites `self.coeffs` again. Also note both `truncate_harmonics` and `prune_harmonics_by_energy` should share the *same* `_original_coeffs` snapshot attribute — don't accidentally give them separate snapshot variables, or

truncation calls could pollute what pruning sees, or vice versa.

**k = 150 sanity check.** Both methods should retain all 301 harmonics and achieve `energy_ratio = 1.0000` exactly, since  $|n| \leq 150$  covers every harmonic that exists.

## Quiz 2 — Shape Transformation via Coefficients

**Rotation derivation.** Rotating every point of the drawing counter-clockwise by  $\theta$  means the new signal is  $g(t) = e^{j\theta} \cdot f(t)$ . Substitute the series for  $f(t)$ :

$$g(t) = e^{j\theta} \sum c_n e^{jn\omega t} = \sum (c_n e^{j\theta}) e^{jn\omega t}$$

So  $g$ 's own coefficients are simply  $c'_n = c_n \cdot e^{j\theta}$  — **every** harmonic gets multiplied by the same fixed unit-magnitude phasor, independent of  $n$ . This is because rotation is a global, time-independent operation on the output value, so it commutes straight through the sum.

**Scaling derivation.** Scaling every point by real factor  $s$  means  $g(t) = s \cdot f(t) = \sum (s \cdot c_n) e^{jn\omega t}$ , so  $c'_n = s \cdot c_n$  for every  $n$  — linearity of the sum again.

```
def get_rotated_coeffs(self, theta):
    phasor = np.exp(1j * theta)
    return {n: c * phasor for n, c in self.coeffs.items()}
```

```
def get_scaled_coeffs(self, factor):
    return {n: factor * c for n, c in self.coeffs.items()}
```

```
def verify_transformation(self, transformed_coeffs, z_reference, t):
    original_coeffs = self.coeffs          # save so we can restore it
    self.coeffs = transformed_coeffs
    f_hat = self.approximate(t)
    self.coeffs = original_coeffs          # restore -- other code still needs the original object
    return np.mean(np.abs(z_reference - f_hat) ** 2)
```

**Why swap-and-restore instead of a temporary FourierEpicycles copy?** Either works, but swapping is cheaper — you avoid re-running `calculate_all_coefficients`, since `approximate` only ever reads `self.coeffs`. Just make sure you restore the dict *unconditionally* (before returning, not after some early-exit path) so later code in your script isn't silently working with the wrong coefficients.

**Why should MSE be tiny for every  $\theta$ /factor?** Both transformations are exact algebraic identities on the coefficients — there is no new integration or approximation introduced. The only error present is whatever error already existed in the original  $c_n$  from the trapezoidal quadrature in `calculate_cn`, propagated through unchanged. If your MSE at  $\theta = 0$  (a no-op rotation) isn't near-zero, the bug is in `verify_transformation`'s bookkeeping, not the math.

## Quiz 3 — Quadrant Masking & Conjugate Symmetry

```
def quadrant_mask(self, real, imag, keep_quadrants):
    rows, cols = real.shape
    ci, cj = rows // 2, cols // 2
    i = np.arange(rows).reshape(-1, 1)
    j = np.arange(cols).reshape(1, -1)
    di = i - ci          # (rows, 1), broadcasts down columns
    dj = j - cj          # (1, cols), broadcasts across rows

    q1 = (di <= 0) & (dj > 0)
    q2 = (di <= 0) & (dj <= 0)
```

```

q3 = (di > 0) & (dj <= 0)
q4 = (di > 0) & (dj > 0)
quadrant_of = {1: q1, 2: q2, 3: q3, 4: q4}

keep_mask = np.zeros((rows, cols), dtype=bool)
for q in keep_quadrants:
    keep_mask |= quadrant_of[q]

real_out = real.copy()
imag_out = imag.copy()
real_out[~keep_mask] = 0
imag_out[~keep_mask] = 0
return real_out, imag_out

```

**Why the four conditions never overlap and never miss a pixel:** each pixel's  $(\Delta i, \Delta j)$  sign pair falls into exactly one of  $\{\leq 0, > 0\} \times \{\leq 0, > 0\}$  — a strict partition of the plane into four quadrants, with the boundary rows/columns ( $\Delta i = 0$  or  $\Delta j = 0$ ) consistently assigned by the  $\leq$  convention so no pixel is double-counted or dropped.

```

def verify_conjugate_symmetry(self, real, imag):
    # F(-u, -v) is the point-reflection of the spectrum through its center,
    # obtained by flipping both axes.
    real_reflected = np.flip(real) # flips both axis 0 and axis 1
    imag_reflected = np.flip(imag)

    # Required: real_reflected == real and imag_reflected == -imag
    real_deviation = np.abs(real_reflected - real)
    imag_deviation = np.abs(imag_reflected - (-imag))

    max_deviation = float(max(real_deviation.max(), imag_deviation.max()))
    is_symmetric = max_deviation < 1e-6
    return is_symmetric, max_deviation

```

**Where this symmetry comes from.**  $I(x,y)$  is real, so in  $F(u,v) = \sum I(x,y) e^{-j2\pi(ux+vy)}$ , taking the complex conjugate of both sides only conjugates the exponential (since  $I$  is already real):  $F(u,v)^* = \sum I(x,y) e^{+j2\pi(ux+vy)} = F(-u,-v)$ . This is exactly why `InverseCFT2D.reconstruct()` is allowed to discard its imaginary output after a symmetric filter (like `high_pass` or `band_pass`) — this quiz asks you to verify the symmetry directly instead of relying on it implicitly.

**Why quadrant-masked reconstructions look asymmetric / have visible artifacts:** keeping only quadrants  $\{1,3\}$  or  $\{1,2\}$  *breaks* the conjugate symmetry the original spectrum had (those masks are not symmetric under  $(u,v) \rightarrow (-u,-v)$ ) — so the reconstructed image picks up a nontrivial imaginary component that `reconstruct()` silently discards, producing visible ringing or directional smearing. This is worth saying out loud if an evaluator asks why these outputs look different from the band-pass/band-stop ones, which stay radially symmetric and therefore stay clean.

## Quiz 4 — Notch Filter & Parseval Energy Check

```

def notch_filter(self, real, imag, center_radius, width):
    rows, cols = real.shape
    cx, cy = rows // 2, cols // 2
    r_low = center_radius - width / 2
    r_high = center_radius + width / 2

    real = real.copy()
    imag = imag.copy()
    for i in range(rows):
        for j in range(cols):
            d = np.sqrt((i - cx) ** 2 + (j - cy) ** 2)
            if r_low <= d <= r_high:

```



```

        real[i, j] = 0
        imag[i, j] = 0
    return real, imag

```

Written loop-style to match the given `high_pass`. Note the boundary is **inclusive on both sides** here ( $\leq \dots \leq$ ), unlike band-pass/band-stop's strict-lower/inclusive-upper convention — the spec for this task explicitly says ' $\leq$ ' on both ends, so match it exactly rather than reusing the other filter's boundary convention from memory.

```

def check_energy_conservation(self, image, real, imag):
    E_spatial = np.sum(image ** 2)
    E_freq_raw = np.sum(real ** 2 + imag ** 2)
    k = E_spatial / E_freq_raw
    return k, None # placeholder -- see full version below

def check_energy_conservation(self, image, real, imag, real_f=None, imag_f=None):
    E_spatial = np.sum(image ** 2)
    E_freq_raw = np.sum(real ** 2 + imag ** 2)
    k = E_spatial / E_freq_raw

    E_freq_filtered = np.sum(real_f ** 2 + imag_f ** 2)
    retained_fraction = (E_freq_filtered * k) / E_spatial
    return k, retained_fraction

```

**Why this is Parseval's theorem, not a made-up formula:** Parseval says energy is preserved between a signal and its transform, up to a fixed scale factor set by how the transform is normalized. Here the CFT was computed with a specific discrete quadrature grid (trapezoidal weights over `self.x/self.y`), so the constant  $k$  isn't the textbook ' $1$ ' or ' $1/(2\pi)^2$ ' you'd see for a purely continuous transform — it depends on your specific  $\Delta x$ ,  $\Delta y$  grid spacing. That's exactly why the task asks you to find  $k$  *empirically* from the unfiltered spectrum rather than derive and hard-code it.

**Sanity check on retained\_fraction:** it must lie in  $[0, 1]$  for any filter that only zeroes entries (never amplifies them) — a notch filter can only remove energy, never add it. If you compute a fraction  $> 1$ , the bug is almost always using the *original* unfiltered spectrum's energy in the numerator instead of the filtered one's.

**Why the retained fraction is close to 1.0 for a narrow notch:** a ring of radius 30, width 10 covers a small fraction of the total (u,v) grid area compared to the whole spectrum, and most images (per Quiz 3's magnitude-spectrum discussion) concentrate most of their energy very close to the DC center — well inside radius 30 for a typical  $256 \times 256$  image — so a mid-radius notch like this only shaves off a small, specific band of mid-frequency content.

---

*End of practice set. Good luck tomorrow — walk through each answer's **derivation**, not just the final code, since that is what an evaluator will actually probe.*